SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**192511061**
Name	**Vidhushini.T.S**

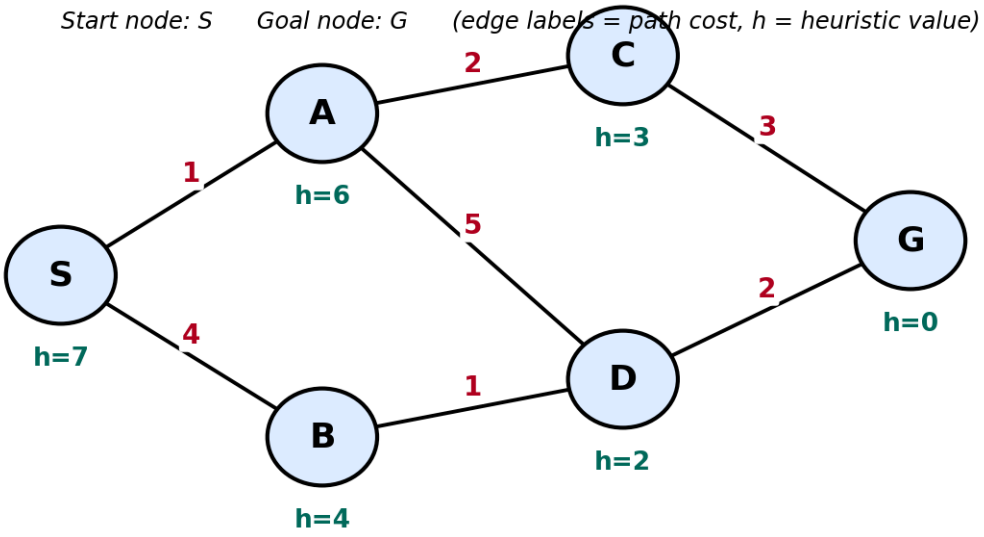
COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

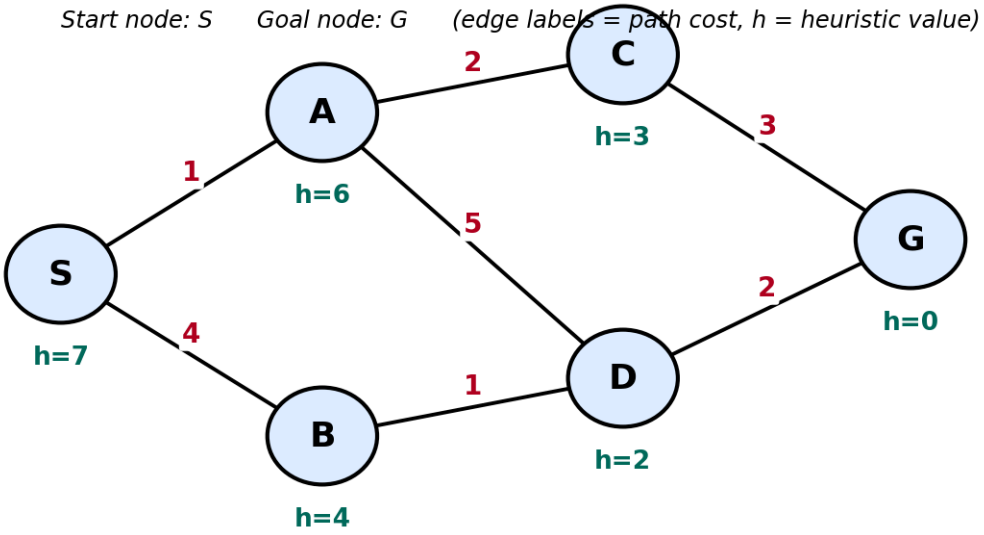
QUESTIONS: 5 **Total Marks:50**

Q1. Consider a suitable state-space graph (with heuristic values h(n) assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.

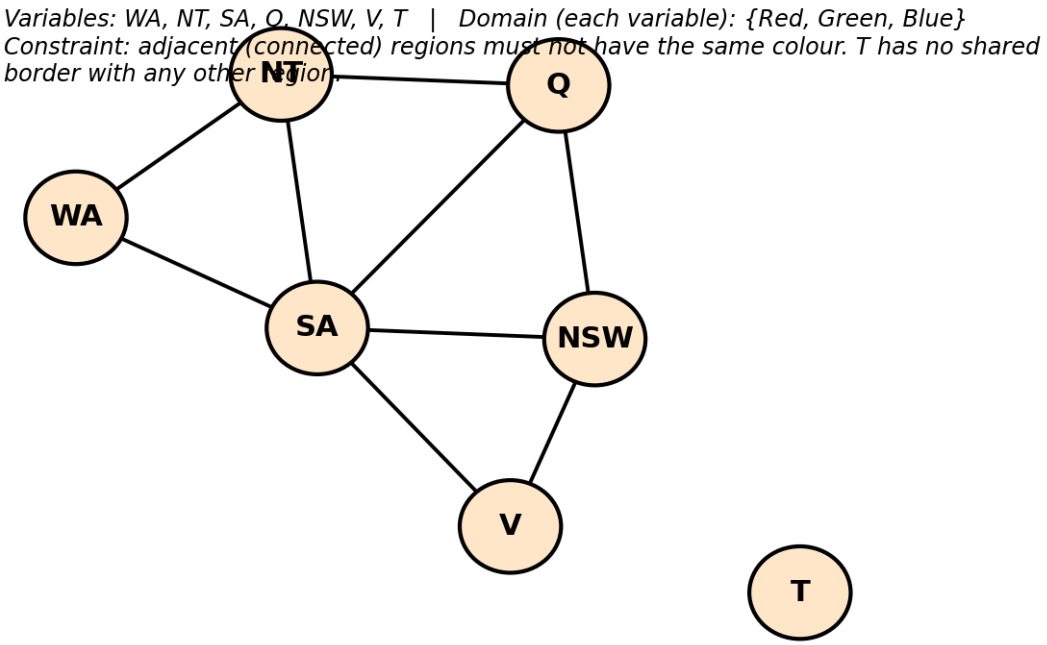


Q2. For a suitable weighted state-space graph (with edge costs and heuristic values h(n)) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the g(n), h(n) and f(n) values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



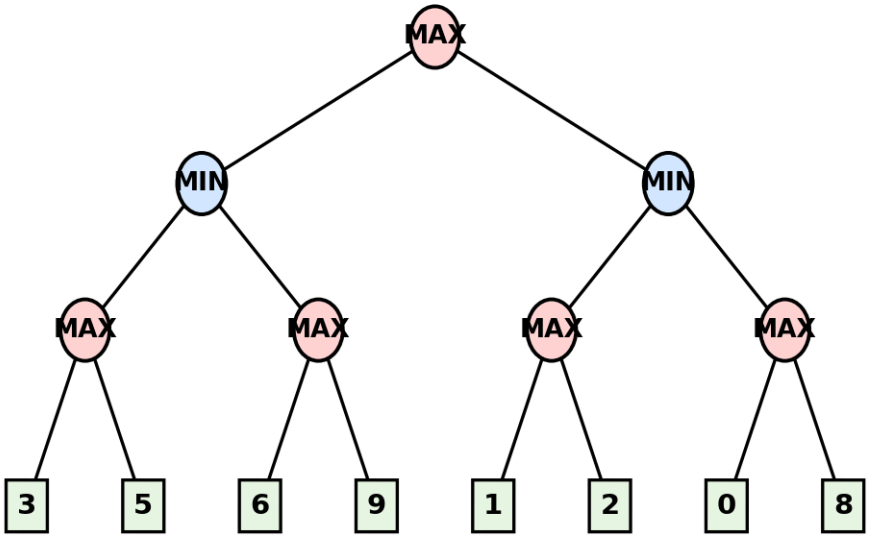
Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.



Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes

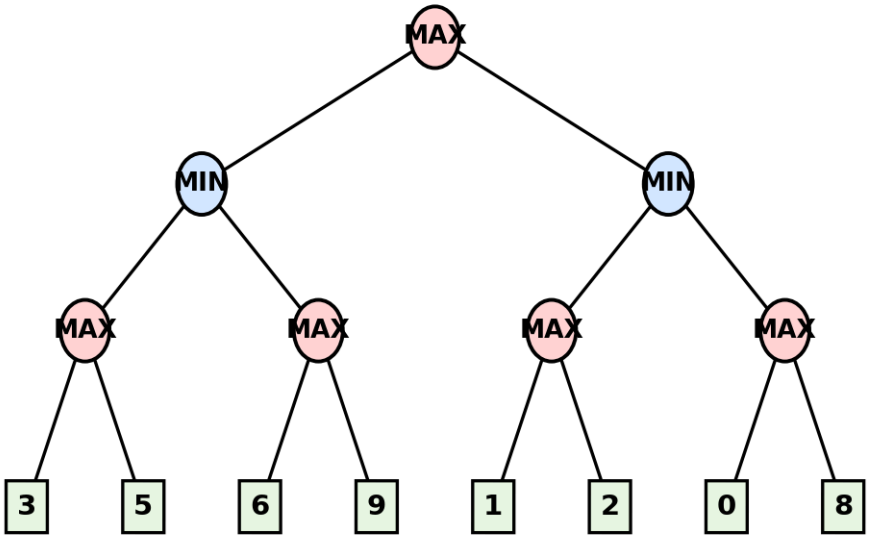
and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



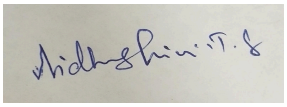
Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I Vidhushini.T.S (192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Dry Run Evaluation

RUBRICS FOR CALCULATION

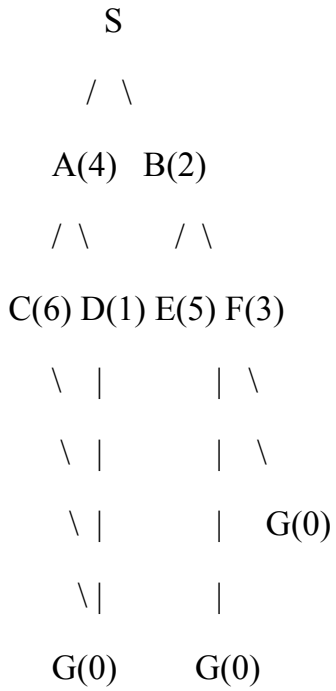
Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	**10**
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	**10**
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly	Multiple calculation errors are present that affect the correctness of	Calculations are mostly incorrect or missing.	**10**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
values, or α–β updates)		affect the final outcome.	intermediate steps.		
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	**10**
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	**10**
				Total	50

Q1. Greedy Best-First Search – Detailed Dry Run

Problem :

Consider the following state-space graph. The heuristic value h(n) represents the estimated distance from a node to the goal.



For simplicity, the useful path is:

S → B → F → G

Heuristic values

Node	h(n)
S	5
A	4
B	2
C	6

D	1
E	5
F	3
G	0

Algorithm :

Greedy Best-First Search selects the node with the lowest heuristic value.

f(n)=h(n)

It does not consider the actual path cost g(n).

Step 1: Start at S

Initial node: S

Expand S.

Its successors are: A(4), B(2)

Compare: h(A)=4 , h(B)=2

Since: 2<4

select B.

Frontier: B(2), A(4)

Step 2: Expand B

Current node: B

Successors: E(5), F(3)

Frontier becomes: A(4), E(5), F(3)

Smallest heuristic:

h(F)=3

Therefore select F.

Step 3: Expand F

Current node: F

F generates: G(0)

Frontier: G(0), A(4), E(5)

Since: h(G)=0

G is selected.

Step 4: Goal reached

G is the goal node.

Therefore the search terminates.

Complete dry-run table

Step	Current Node	Generated Nodes	Frontier / Open List	Selected
1	S	A(4), B(2)	B(2), A(4)	B
2	B	E(5), F(3)	F(3), A(4), E(5)	F
3	F	G(0)	G(0), A(4), E(5)	G
4	G	—	—	Goal

Expansion order : S→B→F→G

Final path

S→B→F→G

Q2. A* Search – Detailed Dry Run

The A* algorithm uses:

$f(n)=g(n)+h(n)$

where:

- g(n) = actual cost from start to current node
- h(n) = estimated cost from current node to goal
- f(n) = estimated total cost

Assume:

S → A = 1

A → C = 2

C → G = 2

C → D = 2

D → G = 1

S → B = 4

Heuristic values

Node	h(n)
S	5
A	4
B	6
C	2
D	1

G	0

Step 1: Expand S

Initially:

g(S)=0, h(S)=5, f(S)=0+5=5

From S: Node A

g(A)=0+1=1, h(A)=4, f(A)=1+4=5

Node B

g(B)=0+4=4, h(B)=6, f(B)=4+6=10

Open list:

Node	g	h	f
A	1	4	5
B	4	6	10

Select A because it has the lowest f.

Step 2: Expand A

A → C has cost 2.

Therefore:

g(C)=g(A)+2 , g(C)=1+2=3

Given:

h(C)=2

Therefore:

f(C)=3+2=5

Open list:

Node	g	h	f
C	3	2	5
B	4	6	10

Select C.

Step 3: Expand C

C generates G and D.

For G

g(G)=3+2=5, h(G)=0, f(G)=5+0=5

For D

g(D)=3+2=5, h(D)=1, f(D)=5+1=6

Open list:

Node	g	h	f
G	5	0	5
D	5	1	6
B	4	6	10

Select G.

Step 4: Goal reached

Since G is the goal, stop.

Expansion order : S→A→C→G

Optimal path : S→A→C→G

Total cost : 1+2+2=5

Optimal Cost=5

Q3. CSP – Map Colouring Using Backtracking

The question requires variables, domains, constraints, assignments, constraint checks and backtracking steps.

Problem formulation

Consider four regions:

Variables :

X={A,B,C,D}

Domain: Each region can have one of three colours:

D(A)=D(B)=D(C)=D(D)={R,G,B}

where:

- R = Red
- G = Green
- B = Blue

Constraints :

Adjacent regions must have different colours.

A≠B, A≠C, B≠C, B≠D, C≠D

Backtracking process

We assign variables in the order:

A→B→C→D

Step 1: Assign A

Try: A=R

No previous assignments exist.

Therefore: A = Red

Step 2: Assign B

First try: B=R

Check: A≠B

But: R=R

Therefore: A ≠ B

So we backtrack and try another colour.

Try:

B=G

Now: R≠G

Therefore:

B = Green

Step 3: Assign C

Try: C=R

Check: A≠C

But: R=R

Try: C=G

Check: A≠C

But: B≠C

Conflict.

Therefore try: C=B

Check: A≠C and: B≠C

Therefore: C = Blue

Step 4: Assign D

Try: D=R

Check: B≠D

Check: C≠D

Therefore:

D = Red

Final solution

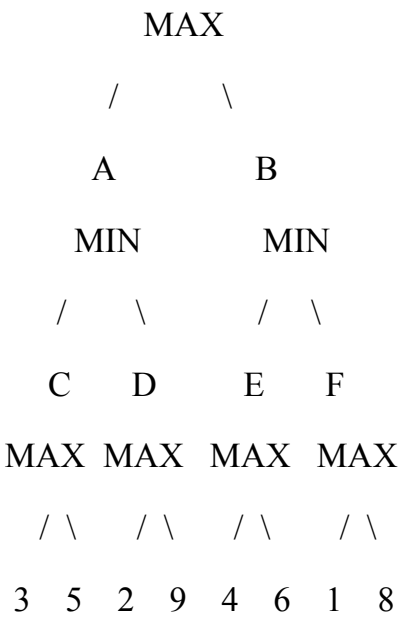
A=R, B=G, C=B, D=R

Q4. Minimax – Detailed Dry Run

Minimax is used for two-player games where:

- MAX tries to maximize the score.
- MIN tries to minimize the score.

Consider the following game tree:



Step 1: Evaluate leaf nodes

Leaf utility values are:

C → 3, 5

D → 2, 9

E → 4, 6

F → 1, 8

Step 2: MAX nodes

Node C

MAX chooses the larger value:

C=max(3,5)=5

Node D

D=max(2,9)=9

Node E

E=max(4,6)=6

Node F

F=max(1,8)=8

So:

C = 5

D = 9

E = 6

F = 8

Step 3: MIN nodes

Node A

MIN chooses the smaller value:

A=min(5,9) , A=5

Node B

B=min(6,8), B=6

Step 4: Root MAX

Root compares:

A=5, B=6

MAX chooses:

max(5,6)=6

Therefore:

Best Move=B

Final answer :

Best move = B Minimax value = 6

Justification

MAX assumes that MIN will always make the best move for itself. Therefore:

Move A → MIN gives 5

Move B → MIN gives 6

MAX chooses B because:

6>5

Q5. Minimax with Alpha-Beta Pruning – Detailed

Alpha-Beta pruning improves Minimax by avoiding branches that cannot influence the final decision.

Two values are maintained:

Alpha : α

The best value that MAX can guarantee so far.

Beta: β

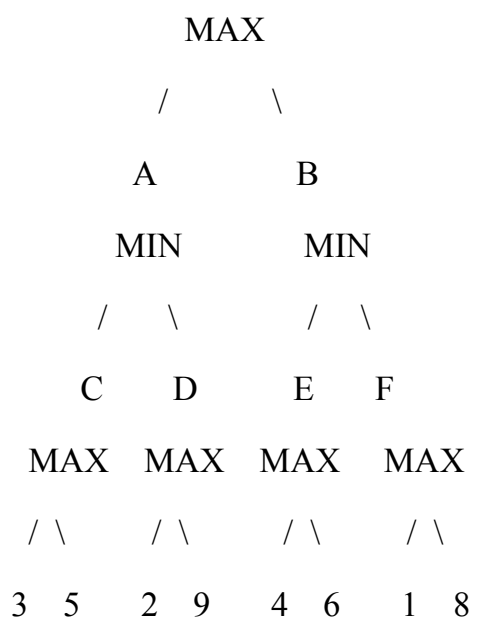
The best value that MIN can guarantee so far.

Pruning condition

A branch can be pruned when:

$\alpha \geq \beta$

Game tree :



Traversal is left to right.

Initial values

At root:

$\alpha = -\infty, \ \beta = +\infty$

Step 1: Explore A

At A: $\alpha=-\infty,\beta=+\infty$

Explore C

C has: 3, 5

Since C is MAX:

$C=\max(3,5)=5$

At A: $\beta=\min(+\infty,5)=5$

So: $\beta A=5$

Explore D

D has: 2, 9

Therefore:

$D=\max(2,9)=9$

At A: $A=\min(5,9)=5$

Return 5 to root.

Step 2: Update root

Root is MAX.

Current value: 555

Therefore:

$\alpha=\max(-\infty,5),\ \alpha=5$

Now explore B.

Step 3: Explore B

At B: $\alpha=5,\ \beta=+\infty$

Explore E.

Node E

E has: 4, 6

Therefore:

E=max(4,6)=6

At B: $\beta=\min(+\infty,6)$, $\beta=6$

Now compare: $\alpha=5$ $\beta=6$

Since: $5<6$

There is no pruning yet.

Step 4: Explore F

F has: 1, 8

Therefore:

F=max(1,8)=8

At B: B=min(6,8) , B=6

Step 5: Root decision

Root has: A = 5, B = 6

Therefore:

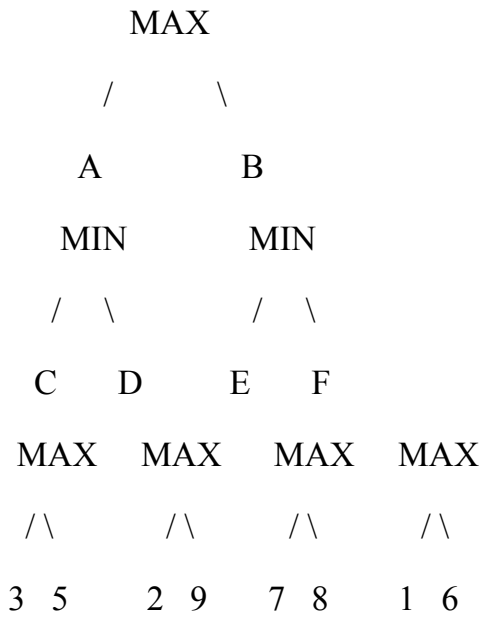
MAX=max(5,6), MAX=6

So: Best Move = B

For this particular left-to-right traversal, there is no actual branch to prune, because the condition $\alpha\geq\beta$ is never reached during a branch expansion.

This is still a valid Alpha-Beta dry run. Alpha-Beta may prune branches depending on the order in which nodes are visited.

Consider:



After exploring A:

A=min(5,9)=5

Thus root gets:

α=5

Now at B, explore E:

E=max(7,8)=8

Thus: βB=8

Still: α=5<β=8

So F cannot yet be pruned.

To force pruning, node ordering and values need to be chosen so that a MIN node gets a value ≤α. For example, if E evaluates to 4:

βB=4

Now: α=5 and: α≥β

Therefore the remaining child F is pruned.